EEEN30160 – Biomedical Signal Processing

Final Project

Oisín Hogan

19355123

Contents

Background information 3
Pan-Thompkins Algorithm 3
Task 1 - Display 4
Output 4
Discussion 4
Task 2 - Filtering 5
Output 5
Discussion 5
Task 3- Segmentation 7
Output 7
Discussion 8
Task 4 and 5 – Classification of PVCs 8
Output 8
Discussion 8
QRS complex 9
R-R period 9
R-Wave Peak 9
Conclusion 9
References 9
Code 10

Background information

Pan-Thompkins Algorithm

The Pan-Thompkins algorithm is a method of identifying heart beats in an ECG signal. This method requires the use of:

1. A band pass filter of frequency 5-15 Hz
2. Derivation of the filter in order to highlight the QRS complex
3. Squaring the signal
4. Averaging of the signal to remove high frequency noise form it
5. Thresholding the signal

This would be done in order to have a segmented signal that has singled out each heartbeat.

Task 1 - Display

Output

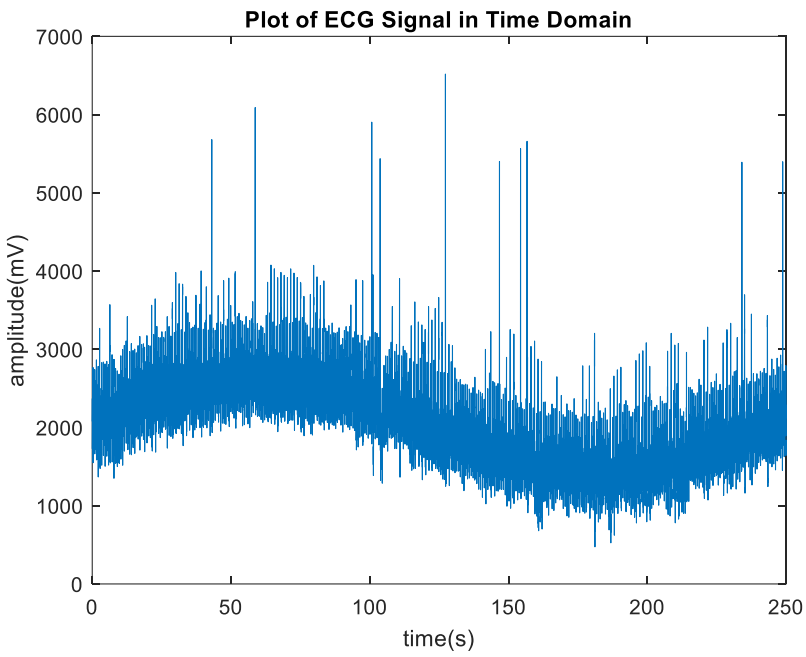

Figure 1

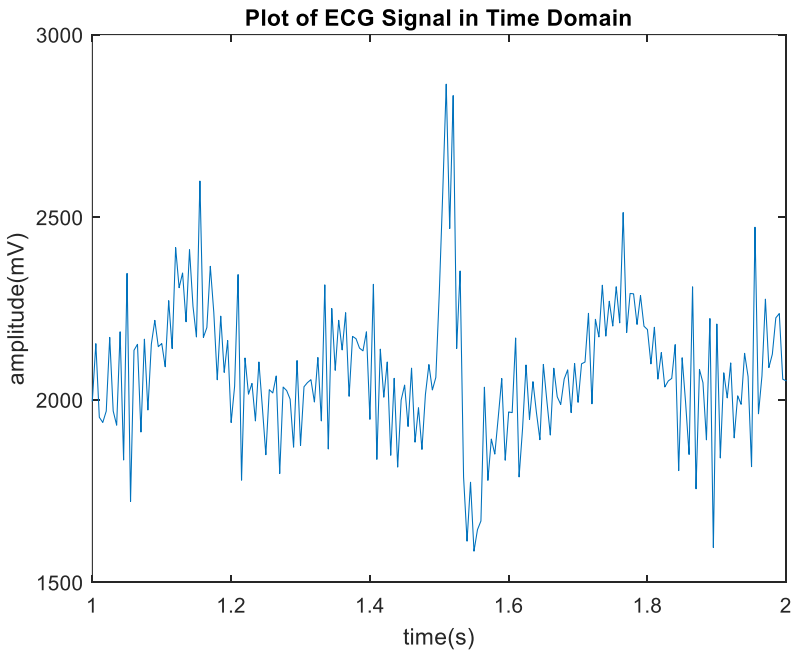

Figure 2

Discussion

The zoomed in graph of the ECG in figure 2 shows that there is a PVC in the wave that can be easily observed, on account of its large amplitude at its peak.

Task 2 - Filtering

Output

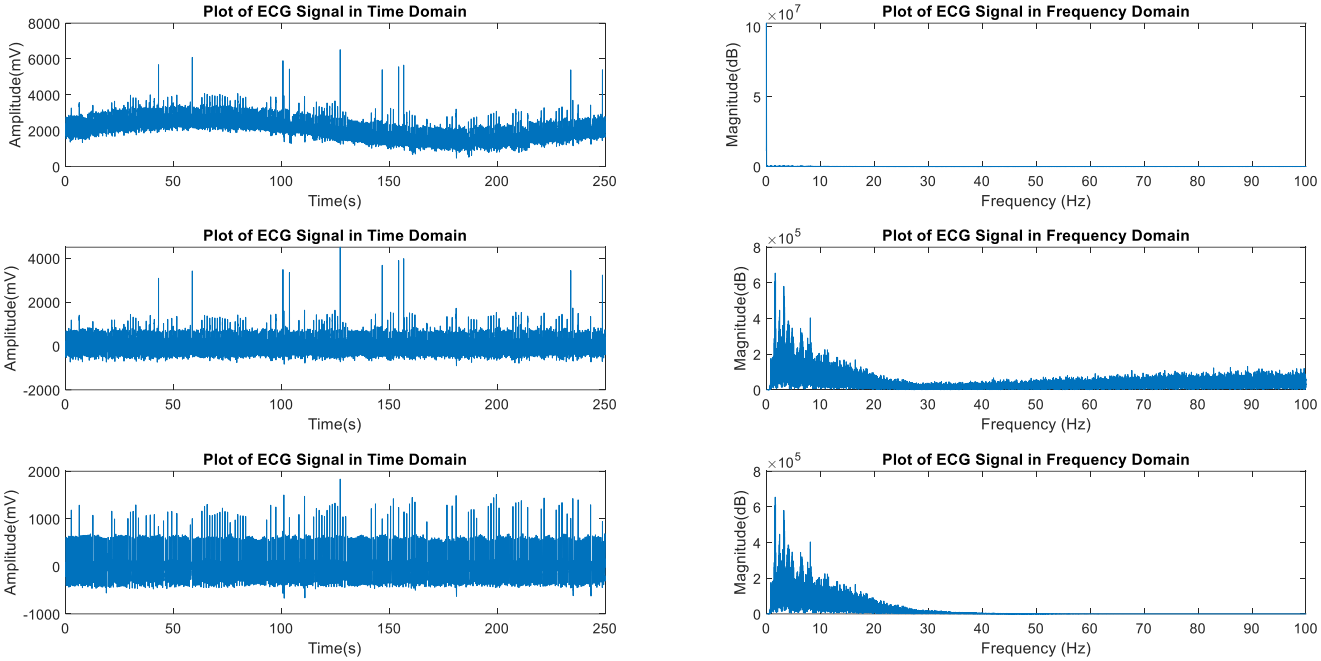

Figure 3

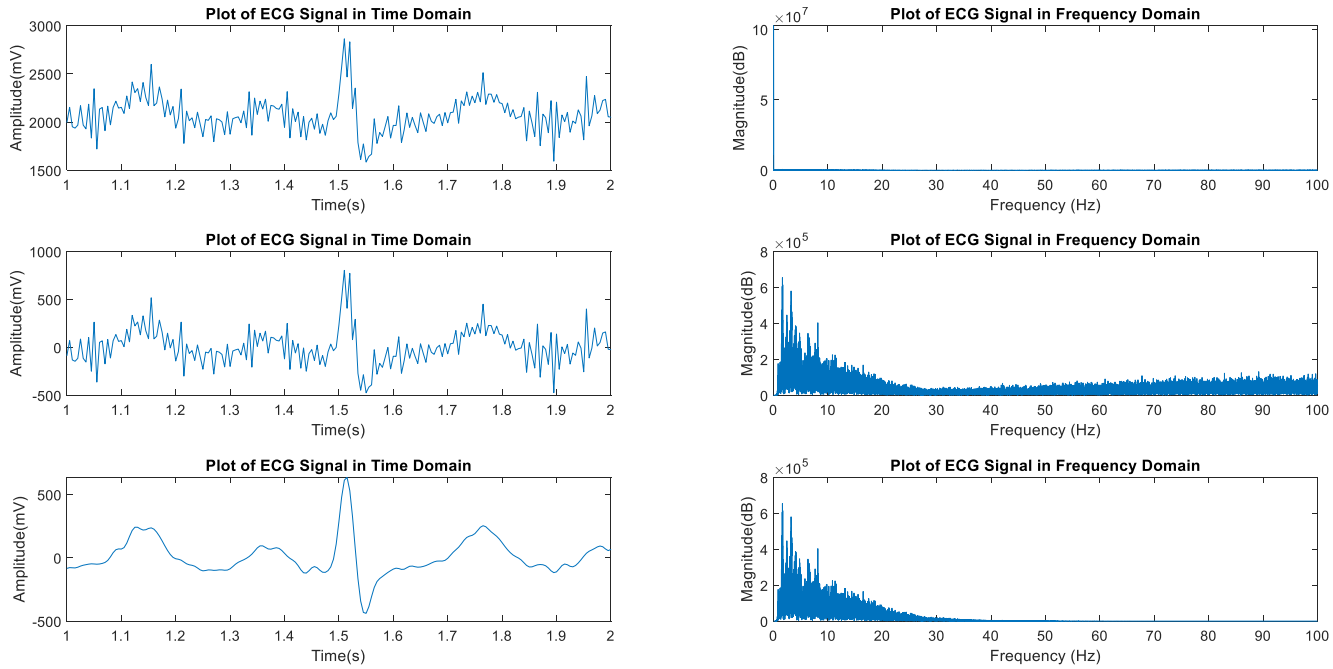

Figure 4

Discussion

From the top left plot of the ECG signal, a baseline wander of the signal can be seen, of approximately 1/250Hz, a cut off frequency of 1 Hz was selected as it would remove this artefact,

this was done with a 3rd order Butterworth high pass filter. This effect can be seen from the change in the signal from the first 2 rows of figure 3.

Looking at the next pair of graphs, once the movement artefact had been removed, there was a lot of high frequency noise on the signal, this was removed with a 3rd order low pass filter, set for 30 Hz low pass. With this done, there was a filtered signal, that could be processed to identify PVCs. The effect of the low pass filtering can be seen in the second 2 rows of figure 4, where there is a zoomed in view of the ECG signal.

Task 3- Segmentation

Output

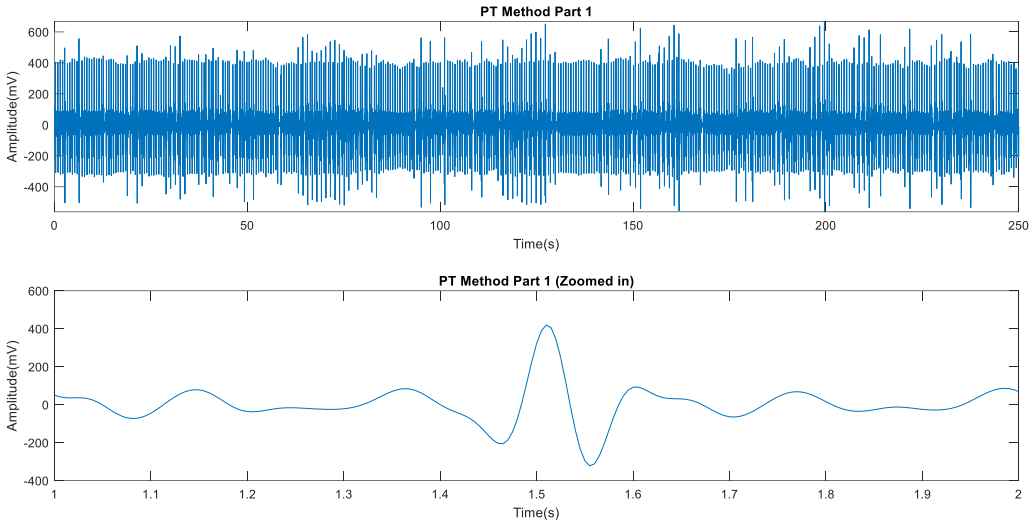

Figure 5

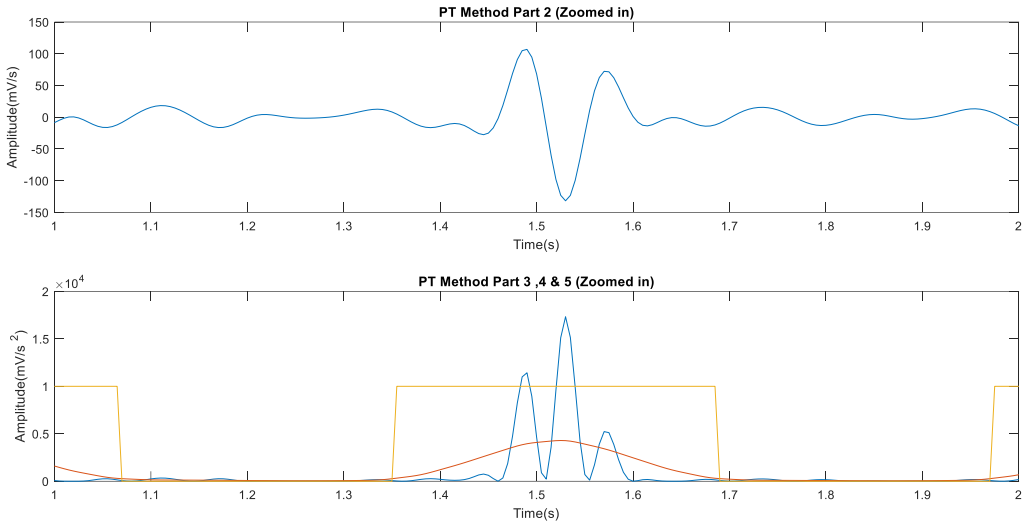

Figure 6

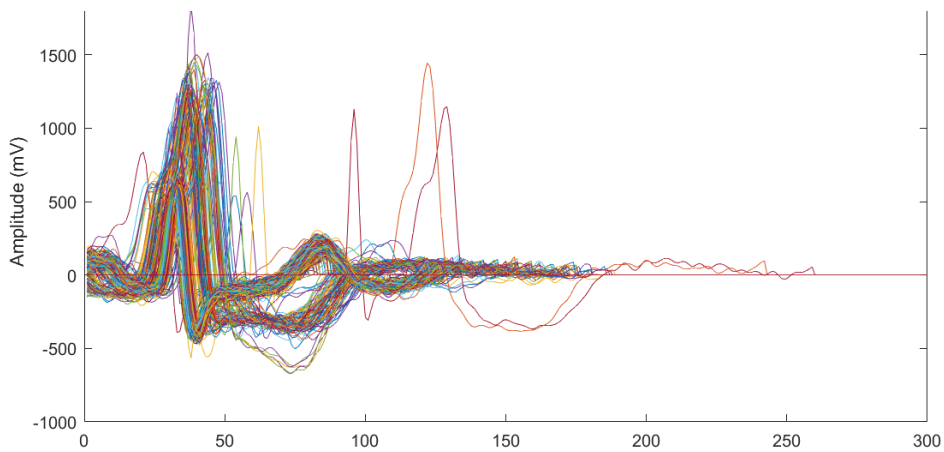

Figure 7

Discussion

In accordance with the Pan-Thompkins method, the signal had been pre-processed, the next step was to utilise a band pass filter at 5 – 15 Hz on the signal, this gave the top 2 graphs in figure 5.

Next the signal had to be differentiated, this was done using the MATLAB function diff(), this provided the first graph in figure 6.

After that the signal was squared, providing the blue line on the next graph of figure 6. An averaging filter of 0.15 seconds was used in order to smooth the signal. This is shown as the smooth, red line in the second graph in figure 6.

The smoothed data was used to find the instances of the ECG where there was a signal. A region of noise was identified between the 1.1 and 1.3 second interval, the mean and standard deviation of the noisy region was found here. This was used to provide the threshold of noise and signal, with experimentation, it was found that 6 standard deviations yielded 399 instances of this occurring which was acceptable.

A code was created to take each segment form the point that the instance was measured and add this to a 'bundle' array. This array was plotted, and the resulting graph is given in figure 7.

Task 4 and 5 – Classification of PVCs

Output

S-Wave Analysis

True Positive	True Negative	False Positive	False Negative	Accuracy
64	278	26	29	86.1460957178841

Table 1

R-R wave interval

True Positive	True Negative	False Positive	False Negative	Accuracy
56	275	29	37	83.3753148614610

Table 2

R–Peak Amplitude

True Positive	True Negative	False Positive	False Negative	Accuracy
66	271	33	27	84.8866498740554

Table 3

Discussion

The 2 methods of identifying the PVCs that were used for the purposes of this were the abnormal QRS complex shape and the compensatory R-R pause, that was associated with the PVC.

QRS complex

From observing figure 7 above, the shape of the PVC S wave can be seen to be either 1 of 2 distinct shapes, where there is a sharp dip, with a rapid rise, and one where there is a gradual rise. Figure 8 below shows that a point was selected between the 2 shapes, this point would be used to classify the difference between the 2 shapes and select the PVCs.

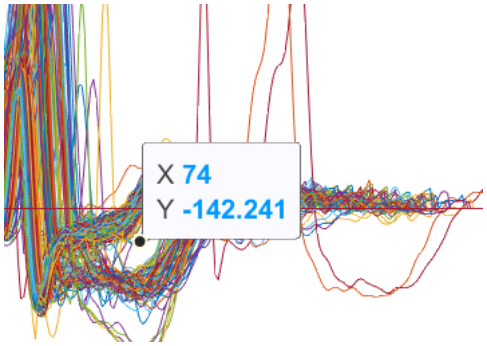

Figure 8

Table 1 above in the results shows the effect that classifying by the S wave was found to have an accuracy of over 86%.

R-R period

The length of each line of the array 'bundles' was counted, and that number was used to classify the length of the epoch. This method of classification yielded the results seen in table 2 above.

R-Wave Peak

When a PVC occurs at a contraction, this can result in an increased amplitude of the R-peak. For this method of classification, a cut off amplitude of 700 mV was selected, and used to drive the classification system, the results of which can be seen in table 3 above.

Figure 9

Conclusion

The 3 methods of identifying PVCs were all of similar efficacy, having an accuracy of 85±2%. The most accurate method of identifying them that was examined in this report was the S-wave analysis which differentiated between each wave on account of the shape that the s- wave of the epoch made. Increased accuracy could possibly be achieved using a Support Vector Machine classifier to combine the different features in order to more accurately classify PVCs.

References

5-3. lesson 5 (cont) ventricular arrhythmias (no date) ECG Learning Center - An introduction to clinical electrocardiography. Available at: https://ecg.utah.edu/lesson/5-3 (Accessed: December 20, 2022).

Sedghamiz, H. (2014) MATLAB implementation of Pan Tompkins ECG QRS detector. - researchgate, Matlab Implementation of Pan Tompkins ECG QRS Detector. Available at: https://www.researchgate.net/profile/Hooman-

Sedghamiz/publication/313673153_Matlab_Implementation_of_Pan_Tompkins_ECG_QRS_detector/data/5ac701130f7e9bcd51932f19/Sedghamiz-2014-Pan-Tompkins-Matlab.pdf (Accessed: December 20, 2022).

Wang, J. et al. (2021) Lead I R-wave amplitude to distinguish ventricular arrhythmias with lead V3 transition originating from the left versus right ventricular outflow tract, Clinical cardiology. U.S. National Library of Medicine. Available at: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC7803356/ (Accessed: December 20, 2022).

Code

```
clc
clear
close all% clear the command windows, workspace and figures

load("G:\My Drive\engineering_work\Biosigs\Project\ecgproject.mat")% load in ecg file
%Part 1
fs = 200;
fn = fs/2;
N = length(ecg1);
t = (0:1:N-1)/fs;% establish constants for the sampling and nyquist frequencies, and the time
domain of the signal
figure()
plot(t, ecg1)
title('Plot of ECG Signal in Time Domain')
xlabel('Time(s)')
ylabel('Amplitude(mV)')
%xlim([1,2])

%part 2 - filtering
ecgang1 = fft(ecg1);
ecgang1 = fftshift(abs(ecgang1));
f = (-N/2:1:N/2-1)/N;
f = f*fs;% create frequency domain array and a fourrier transform of the ECG signal to the
frequency domain

figure()% create a new figure for the output of task 2
tiledlayout(3,2)
nexttile
plot(t, ecg1)
title('Plot of ECG Signal in Time Domain')
xlabel('Time(s)')
ylabel('Amplitude(mV)')
%xlim([1,2])
nexttile
plot(f, ecgang1)
title('Plot of ECG Signal in Frequency Domain')
xlabel('Frequency (Hz)')
ylabel('Magnitude(dB)')
xlim([0, fs/2])

fcolow = 1;% choose high pass cut off frequency
wh = fcolow/fn; %] calcualte the adjusted frequency
[b,a] = butter(3, wh, 'high'); % get constants, 'a' and 'b'

ecg2 = filtfilt(b, a, ecg1); % Filter ecg
ecgang2 = abs(fft(ecg2)); %get the frequencies of filtered ecg
ecgang2 = fftshift(ecgang2);

nexttile
plot(t, ecg2)
title('Plot of ECG Signal in Time Domain')
xlabel('Time(s)')
ylabel('Amplitude(mV)')
%xlim([1,2])
nexttile
plot(f, ecgang2)
title('Plot of ECG Signal in Frequency Domain')
xlabel('Frequency (Hz)')
ylabel('Magnitude(dB)')
xlim([0, .5*fs]) %plot the filtered ecg in the time and frequency domains

fcohigh  = 30; %set the frequency for the lowpass filter
wl = fcohigh/fn;
[b,a] = butter(3, wl);
ecg3 = filtfilt(b,a, ecg2);
ecgang3 = fft(ecg3);
```

```
ecgang3 = fftshift(abs(ecgang3)); % filter the ecg and obtain the fourier transform of the
signal

nexttile
plot(t, ecg3)
title('Plot of ECG Signal in Time Domain')
xlabel('Time(s)')
ylabel('Amplitude(mV)')
%xlim([1,2])
nexttile
plot(f, ecgang3)
title('Plot of ECG Signal in Frequency Domain')
xlabel('Frequency (Hz)')
ylabel('Magnitude(dB)')
xlim([0, fs/2]) % plot final filtered signal in the time and frequency domain

%Part 3 - Segmentation
fband = [5, 15]/fn; % for the first part of the PT method, the filter pass band was
established
[b,a] = butter(3, fband);
ecg4 = filtfilt(b, a, ecg3);% filter ecg with the passband frequencies

figure()
tiledlayout(2,1)
nexttile
plot(t, ecg4)
title('PT Method Part 1')
xlabel('Time(s)')
ylabel('Amplitude(mV)')
nexttile
plot(t, ecg4)
xlim([1,2])
title('PT Method Part 1 (Zoomed in)')
xlabel('Time(s)')
ylabel('Amplitude(mV)')% create a new figure to plot the filtered signal as a whole and zoomed
in

ecg5 = diff(ecg4);
dt = t(1:N-1); % differentiate the filtered signal

figure()
tiledlayout(2, 1)
nexttile
plot(dt, ecg5)
title('PT Method Part 2 (Zoomed in)')
xlabel('Time(s)')
ylabel('Amplitude(mV/s)')
xlim([1,2])%plot zoomed in differentiated signal

nexttile
ecg6 = ecg5.^2;% square signal
plot(dt, ecg6)
title('PT Method Part 3 ,4 & 5 (Zoomed in)')
xlabel('Time(s)')
ylabel('Amplitude(mV/s ^2)')
xlim([1,2]) % plot zoomed in squared signal
hold on

fsa = round(fs*.15); % set adjusted frequency for averaging each .15 seconds of the ecg
coeffavg = ones(1, fsa)/fsa; % create 'b' coefficient for the filtering of the signal
ecg7 = filtfilt(coeffavg, 1, ecg6); %filter the signal
plot(dt, ecg7)% plot the filtered signal over the other signal

noisesig = ecg7(1.1*fs:1:1.3*fs); % create an array of the signal where there is only noise

threshmean = mean(noisesig); % find the mean of the noise
threshsd = std(noisesig);% find the standard deviation of the noise
thresh = threshmean+6*threshsd; % experimentally it was foudnd that 6 standard deviations of
the mean yielded 399 intances of signals being caught
%this was the threshold of amplitude
gorl = zeros(1, N-1);%create an array to see if the threshold has been passed or not
for i = (1:N-2)
    if ecg7(i)>thresh
        gorl(i) = 1;
    end
end % for loop and if statement to fill out gorl array
insts = find((diff(gorl))>0);% find the intances where the threshold is passed, in samples
plot(dt, gorl*10^4) % plot this data on the same graph as the parts 3 and 4
```

```
n = length(insts);
bw = 300;
bundle = zeros(bw, n-1);
%bundle = zeros(bw, n);
for j = 1:n-1
    for i = 1:bw
        if i +insts(j)<insts(j+1)
            bundle(i, j) = ecg3(i +insts(j));
        else
            break
        end
    end
end

figure()
plot(bundle)
title('Bundles of heart beats')
xlabel('Samples')
ylabel('Amplitude (mV)') %Plot each epoch on the same axis in order to identify differences
between the PVC and normal contraction

%step 4 and 5 - Classification

%method 1- S wave analysis
compsampno = 74; %select time for amplitude to be measured on
ampthresh = -140; % set cut off amplitude
tfswave = zeros(n-2, 1); %create a 0's array for results
for i = 1:n-2
   if bundle(compsampno, i)<ampthresh
       tfswave(i) = 1;
   end
end % for loop to generate answer set
% compare results
tp = 0;
tn = 0;
fp = 0;
fn = 0; % initialise the final answer variables
for i = 1:length(tfswave)
    if tfswave(i)== 1&& isPVC(i) ==1
        tp = tp+1;
    end
    if tfswave(i)== 1&& isPVC(i) ==0
        fp = fp+1;
    end
    if tfswave(i)== 0&& isPVC(i) ==0
        tn = tn+1;
    end
    if tfswave(i)== 0&& isPVC(i) ==1
        fn = fn+1;
    end
end %work out the amount of true and false positive and negative answers
acc = (tp+tn)*100/ (tp+tn+fp+fn); % calcualte accuracy
resultssw = {'True Positive', 'True Negative', 'False Positive', 'False Negative', 'Accuracy'
;tp, tn, fp, fn, acc};% create a dataframe of the results of this method

%method 2 R-R length
rrlegth = 140;% select a cut off length of 140 samples
tfrlen = zeros(n-2, 1);
for i = 1:n-2
   len = max(find(bundle(:, i)));% calculate the length of the line in question
   line = bundle(1:len, i); %take out the nonzero numbers from the line
   if length(line)>rrlegth
       tfrlen(i) = 1;
   end
end
% compare results
tp = 0;
tn = 0;
fp = 0;
fn = 0;
for i = 1:length(tfrlen)
    if tfrlen(i)== 1&& isPVC(i) ==1
        tp = tp+1;
    end
    if tfrlen(i)== 1&& isPVC(i) ==0
        fp = fp+1;
    end
    if tfrlen(i)== 0&& isPVC(i) ==0
```

```
        tn = tn+1;
    end
    if tfrlen(i)== 0&& isPVC(i) ==1
        fn = fn+1;
    end
end
acc = (tp+tn)*100/ (tp+tn+fp+fn);
resultsrl = {'True Positive', 'True Negative', 'False Positive', 'False Negative', 'Accuracy'
;tp, tn, fp, fn, acc};% fill out results array in the same way as in method 1

%method 3- R Peak Amplitude Analysis
ampthresh = 700;% set an amplitude threshold of 700 mV
rpawave = zeros(n-2, 1);
for i = 1:n-2
   maxamp = max(bundle(:, i));% find the maximum peak value in each epoch
   if maxamp>ampthresh
       rpawave(i) = 1;
   end
end
% compare results
tp = 0;
tn = 0;
fp = 0;
fn = 0;
for i = 1:length(rpawave)
    if rpawave(i)== 1&& isPVC(i) ==1
        tp = tp+1;
    end
    if rpawave(i)== 1&& isPVC(i) ==0
        fp = fp+1;
    end
    if rpawave(i)== 0&& isPVC(i) ==0
        tn = tn+1;
    end
    if rpawave(i)== 0&& isPVC(i) ==1
        fn = fn+1;
    end
end
acc = (tp+tn)*100/ (tp+tn+fp+fn);
resultsra = {'True Positive', 'True Negative', 'False Positive', 'False Negative', 'Accuracy'
;tp, tn, fp, fn, acc};%cacluate results in the same method as method 1 and 2
```